

# RESEARCH PAPER QUESTION ANSWERING SYSTEM

## USING RETRIEVAL-AUGMENTED GENERATION (RAG)

*A Technical Report*

|                         |                            |
|-------------------------|----------------------------|
| <b>Name</b>             | Talla Tejaswini            |
| <b>Registration No.</b> | 231FA23034                 |
| <b>Branch</b>           | B.Tech — Data Science (DS) |
| <b>Year</b>             | 4th Year                   |
| <b>Project Type</b>     | AI-Powered RAG Application |
| <b>Date</b>             | September 2026             |

Stack: Python · Streamlit-ready · FAISS · Sentence-Transformers · Gemini / OpenAI  
Target Models: all-MiniLM-L6-v2 (embeddings) & Gemini (generation)

# 1. Abstract

---

Researchers today face an overwhelming volume of scientific literature, and manually scanning long PDFs for a specific objective, dataset, or result is slow and error-prone. Directly querying a general-purpose large language model (LLM) about a paper it has never seen is worse: the model tends to hallucinate plausible-sounding methods, numbers, and citations, offers no way to trace an answer back to a source page, and — if the paper is confidential — pushing its full contents to a third-party API raises real privacy concerns.

This report documents a Research Paper Question-Answering System built around Retrieval-Augmented Generation (RAG). The system lets a user upload one or more research papers in PDF format and ask natural-language questions about their contents — objective, methodology, datasets, findings, limitations, and conclusions. Instead of asking the LLM to answer from memory, the pipeline first extracts page-tagged text with PyMuPDF, splits it into overlapping word-window chunks, embeds each chunk with the Sentence-Transformers model all-MiniLM-L6-v2 (384-dimensional, L2-normalised vectors), and indexes them in a FAISS IndexFlatIP store so that inner-product search doubles as cosine-similarity search. At query time, the top-K most relevant chunks are retrieved and assembled into a strictly grounded prompt that is sent to an LLM (Gemini, with an OpenAI fallback) for answer generation. Every answer returned to the user is accompanied by the source paper name, the page range it was drawn from, and the similarity score, so the answer is verifiable rather than taken on faith.

# 2. Introduction

---

Reading a technical paper end-to-end just to answer one question — "what dataset did they use?", "what were the limitations?" — wastes a researcher's time. General-purpose LLMs promise to shortcut this, but applying them directly to an unseen manuscript runs into a few well-known obstacles:

- Parametric limits — a standalone LLM has no knowledge of a paper that was never in its training data, especially unpublished pre-prints or very recent work.
- Hallucination — asked to summarise a document it cannot actually see, an LLM will often fabricate confident-sounding metrics, methods, and even citations.
- Context and "lost-in-the-middle" effects — pasting an entire multi-page PDF into a single prompt is expensive and pushes relevant details into the middle of a long context, where models are known to attend to them less reliably.
- Provenance and trust — even a correct-sounding answer is unusable for serious academic work unless it can be traced back to a specific page.

Retrieval-Augmented Generation addresses all four by decoupling retrieval from generation: the paper's content is embedded and indexed once, offline, and only the handful of chunks that are semantically closest to a given question are pulled in as context for the LLM at query time. This keeps answers grounded in the actual document and keeps every claim traceable to a page.

# 3. Problem Statement

---

- Information overload — long, dense research papers make it slow to locate a specific formula, metric, or method by manual reading.
- Provenance deficit — generic AI chat tools give answers with no exact page reference, so every claim has to be manually re-verified against the PDF.
- Hallucination risk — an LLM queried without grounding will confidently answer questions it has no real basis for.

- Keyword-search limits — plain keyword or Ctrl-F search fails whenever a question is phrased differently from the paper's own wording.

## 4. Objectives

---

- Page-aware extraction — parse the uploaded PDF page by page with PyMuPDF so every unit of text keeps a 1-based page number.
- Configurable chunking — split page text into overlapping, word-count-based sliding-window chunks that keep a coherent idea intact and track the page range each chunk spans.
- Semantic embedding — encode every chunk (and every incoming query) into a 384-dimensional dense vector with Sentence-Transformers' all-MiniLM-L6-v2.
- Fast, accurate retrieval — index chunk embeddings in a FAISS IndexFlatIP store and retrieve the top-K nearest neighbours by cosine similarity on normalised vectors.
- Grounded generation with citations — construct a strict, context-only prompt for the LLM (Gemini, or OpenAI as a drop-in alternative) that forces the model to answer only from retrieved text, or explicitly say the paper does not address the question.
- Transparent, defensible pipeline — implement every stage from first principles (no opaque black-box chain) so each step of retrieval and generation can be explained and inspected.

## 5. Scope of the Project

---

### 5.1 In Scope

- Upload and parsing of one or more research-paper PDFs.
- Word-window chunking with tunable chunk size and overlap.
- Dense semantic embedding and FAISS-based similarity search.
- Context-grounded answer generation with page-and-paper-level source citation.
- A live experiment mode for comparing chunk size, overlap, and retrieval depth (Top-K).

### 5.2 Current Exclusions

- OCR for scanned, non-searchable raster PDFs.
- Cross-paper synthesis beyond simple multi-paper indexing (no dedicated comparison/summary layer).
- User authentication, multi-user accounts, or a persistent hosted deployment.

## 6. System Architecture

---

The system follows a two-phase pipeline: an offline Indexing phase that runs once per uploaded paper, and an online Query phase that runs once per question. Both phases share the same embedding model so that queries and document chunks land in the same vector space.

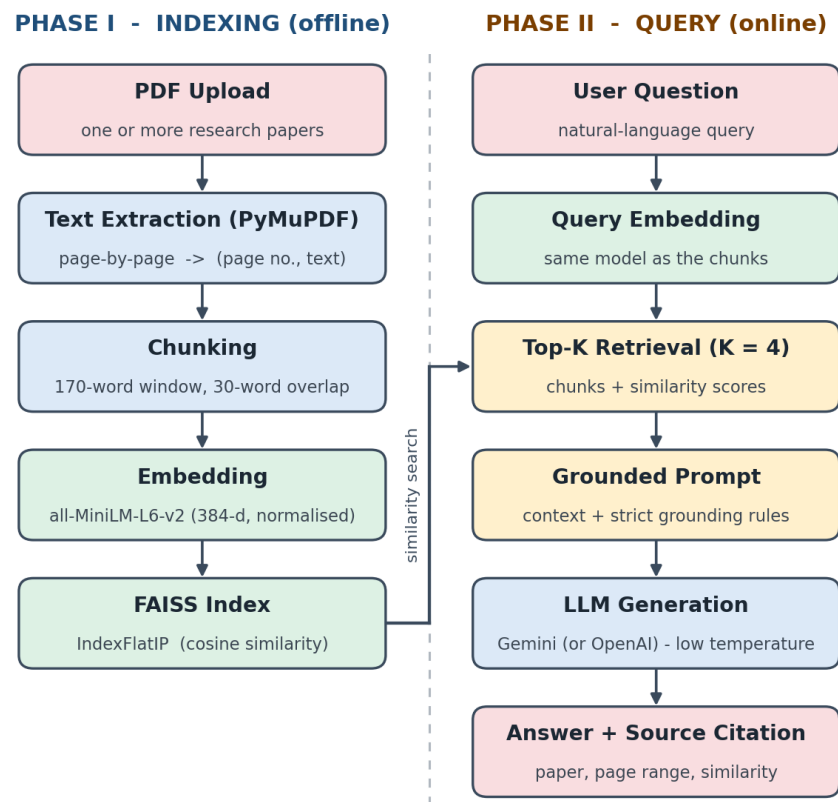


Figure 1. End-to-end RAG architecture — Phase I (offline indexing) and Phase II (online query).

### 6.1 Phase I — Indexing (offline)

- **PDF Upload** — one or more research papers are added to the working set.
- **Text Extraction (PyMuPDF)** — every page is read into a (page number, text) pair, preserving order.
- **Chunking** — page text is flattened into a single word stream and split with a sliding window (a configurable window size and overlap, tuned per experiment) so each chunk keeps its originating page range.
- **Embedding** — each chunk is encoded with all-MiniLM-L6-v2 into a 384-dimensional, L2-normalised vector.
- **FAISS Index** — normalised embeddings are added to a FAISS IndexFlatIP index, so inner-product search is equivalent to cosine-similarity search.

### 6.2 Phase II — Query (online)

- **User Question** — a natural-language question is submitted.
- **Query Embedding** — the question is encoded with the same all-MiniLM-L6-v2 model used for the chunks.
- **Top-K Retrieval (K = 4 by default)** — the FAISS index returns the K chunks with the highest cosine similarity, each tagged with its paper name, page range, and score.
- **Grounded Prompt** — the retrieved chunks are formatted into numbered source blocks and combined with strict grounding instructions (answer only from context; say so explicitly if the context is insufficient; cite the source tag for every claim).
- **LLM Generation** — the prompt is sent to Gemini (with OpenAI GPT-4o-mini as an alternative provider) at low temperature to keep answers consistent and on-topic.

- Answer + Source Citation — the final answer is returned to the user along with the paper name, page range, and similarity score of every chunk that contributed to it.

## 7. Functional Modules

The notebook implementation is organised into six logical stages, each isolated into its own cell/function so it can be tested, explained, and re-run independently:

- PDF Ingestion — accepts uploaded PDFs and stores them for processing; each paper is tracked by filename so multi-paper citations remain unambiguous.
- Page-Aware Extraction — `extract_pages()` uses PyMuPDF (`fitz`) to return 1-indexed (`page_number`, `text`) tuples per document.
- Sliding-Window Chunking — `chunk_paper()` flattens page text into a word stream and slides a fixed-size window forward by (`chunk_size` – `overlap`) words at a time, recording `chunk_id`, `page_start`, and `page_end` for every chunk.
- Dense Vector Encoding & Indexing — `build_index()` encodes all chunk texts with `SentenceTransformer("all-MiniLM-L6-v2")` (normalised embeddings) and adds them to a `faiss.IndexFlatIP` store.
- Semantic Retrieval — `retrieve()` encodes the incoming query the same way and calls `index.search()` to fetch the Top-K nearest chunks with their similarity scores.
- Grounded Generation & Citation — `build_prompt()` assembles a numbered, source-tagged context block and a strict system instruction; `call_llm()` sends it to Gemini (or OpenAI) and returns the answer plus the sorted set of cited pages.

## 8. Key Implementation Snippets

### 8.1 Page-Aware PDF Extraction

```
import fitz # PyMuPDF

def extract_pages(pdf_path):
    doc = fitz.open(pdf_path)
    pages = []
    for i, page in enumerate(doc):
        text = page.get_text("text")
        pages.append((i + 1, text))
    doc.close()
    return pages
```

### 8.2 Sliding-Window Chunking with Page Tracking

```
def chunk_paper(pages, chunk_size, overlap):
    word_page_pairs = [(w, p) for p, t in pages for w in t.split()]
    chunks, step, i, chunk_id = [], max(chunk_size - overlap, 1), 0, 0
    while i < len(word_page_pairs):
        window = word_page_pairs[i:i + chunk_size]
        words = [w for w, _ in window]
        pages_ = [p for _, p in window]
        chunks.append({
            "chunk_id": chunk_id, "text": " ".join(words),
            "page_start": min(pages_), "page_end": max(pages_),
        })
        i += step
        chunk_id += 1
```

```

    chunk_id += 1; i += step
return chunks

```

### 8.3 Embedding + FAISS Cosine Index

```

embed_model = SentenceTransformer("all-MiniLM-L6-v2")

def build_index(chunks):
    embeddings = embed_model.encode(
        [c["text"] for c in chunks],
        normalize_embeddings=True)
    index = faiss.IndexFlatIP(embeddings.shape[1])
    index.add(embeddings.astype("float32"))
    return index, embeddings

```

### 8.4 Grounded Prompt Construction

```

def build_prompt(question, retrieved_chunks):
    blocks = [f"[Source {i}: {c['paper']}], "
              f"p.{c['page_start']}-{c['page_end']}]\\n{c['text']}"
              for i, c in enumerate(retrieved_chunks, 1)]
    context = "\\n\\n".join(blocks)
    return ("Answer strictly from the CONTEXT below. If it is not "
            "enough, say so. Cite the source tag for every claim.\\n\\n"
            f"CONTEXT:\\n{context}\\n\\nQUESTION: {question}")

```

## 9. Workflow and Data Flow

- Phase I (Indexing): PDF upload → `extract_pages()` extracts page-tagged text → `chunk_paper()` builds overlapping word-window chunks → `build_index()` embeds chunks and writes them into a FAISS IndexFlatIP store.
- Phase II (Query & RAG): User question → the question is embedded with the same model → FAISS cosine search retrieves the Top-K most relevant chunks → `build_prompt()` builds a grounded, source-tagged prompt → `call_llm()` queries Gemini/OpenAI → the answer is returned with its cited pages.

## 10. Technology Stack and Justification

| Technology            | Role             | Why It Was Chosen                                                                               |
|-----------------------|------------------|-------------------------------------------------------------------------------------------------|
| Python 3.10+          | Primary language | Mature AI/ML ecosystem with first-class bindings for FAISS, PyMuPDF, and Sentence-Transformers. |
| PyMuPDF (fitz)        | PDF parser       | High-speed, C-based text extraction that preserves page structure for citation.                 |
| Sentence-Transformers | Dense encoder    | all-MiniLM-L6-v2 gives a strong speed/accuracy trade-off at only 384 dimensions.                |
| FAISS (CPU)           | Vector index     | IndexFlatIP on normalised vectors gives exact, sub-millisecond cosine-similarity search.        |
| Gemini API            | Primary LLM      | Free-tier, low-latency generation used for the grounded-answer step.                            |
| OpenAI (GPT-4o-mini)  | Fallback LLM     | Drop-in alternative provider, selected via a single LLM_PROVIDER config flag.                   |

## 11. Configuration Parameters

The pipeline exposes its retrieval knobs as explicit configuration values rather than hard-coding them, which makes the trade-offs easy to demonstrate and tune:

| Parameter             | Role                                         | Trade-off                                                                                                 |
|-----------------------|----------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Chunk size (words)    | Size of each retrieval unit                  | Small → precise but loses surrounding context. Large → more context but noisier, less-focused embeddings. |
| Chunk overlap (words) | Shared boundary text between adjacent chunks | Prevents a sentence that straddles a chunk boundary from being lost in both halves.                       |
| Top-K                 | Retrieval depth passed to the LLM            | Too low misses relevant evidence; too high dilutes the prompt and raises hallucination/cost risk.         |
| Embedding model       | all-MiniLM-L6-v2, 384-d                      | Strong CPU-friendly speed/accuracy balance for interactive use.                                           |
| LLM provider          | Gemini (default) / OpenAI                    | Swappable via a single config flag with no other code changes.                                            |

*A dedicated `experiment()` routine rebuilds the index at different chunk sizes, overlaps, and Top-K values and prints the retrieved chunks side by side, so the effect of each parameter on retrieval quality can be shown directly rather than argued from theory alone.*

## 12. Strengths of the System

- Grounded answers — the strict context-only prompt and explicit "paper does not address this" fallback sharply reduce hallucination.
- Verifiable provenance — every answer is traceable to a paper, page range, and similarity score, not just a plain-text claim.
- Semantic, not keyword, retrieval — questions worded differently from the paper's own language still retrieve the right chunks.
- Deterministic citation mapping — source tags are generated by code, not guessed by the LLM, so citations cannot be hallucinated.
- Transparent, from-scratch pipeline — no opaque framework chain; every stage (extraction, chunking, embedding, retrieval, prompting) is an inspectable function.
- Multi-paper ready — chunk metadata already carries a paper field, so the same FAISS index can hold and disambiguate several documents at once.

## 13. Limitations and Challenges

- Fixed-size chunking can still split a formula, table, or long sentence across a chunk boundary despite the overlap.
- Top-K retrieval may miss an answer whose supporting evidence is spread across more pages than K chunks can cover.
- Text-only extraction — the system does not perform OCR, so scanned or purely image-based PDFs are not supported.

- IndexFlatIP is an exact search; it is fast for a single paper but does not scale to a very large multi-paper corpus without moving to an approximate index (e.g. IVF/HNSW).
- Depends on an external LLM API (Gemini/OpenAI) for the generation step, so it is not fully offline.

## 14. Future Enhancements

- Hybrid retrieval — combine FAISS vector search with BM25 keyword matching via Reciprocal Rank Fusion for stronger recall on rare technical terms.
- Re-ranking — add a cross-encoder re-ranker (e.g. bge-reranker-large) to re-score the top candidates before they reach the LLM.
- Multi-paper library mode — extend metadata filtering so a user can restrict retrieval to one paper within a larger indexed library.
- OCR and multi-modal parsing — integrate Tesseract OCR and layout-aware parsing to cover scanned papers, tables, and figures.
- Approximate-nearest-neighbour indexing — move from IndexFlatIP to IndexIVFFlat/HNSW, or a managed vector database, for large-scale deployments.

## 15. Installation and Usage Guide

```
# 1. Install dependencies
pip install pymupdf sentence-transformers faiss-cpu google-generativeai openai

# 2. Set an LLM API key
export GEMINI_API_KEY=your_key_here      # or OPENAI_API_KEY

# 3. Run the pipeline / notebook
#   - upload one or more PDFs
#   - the notebook indexes them automatically
#   - ask questions in the interactive Q&A cell
```

## 16. Dependencies

| Package               | Role                                      |
|-----------------------|-------------------------------------------|
| pymupdf (fitz)        | PDF text extraction                       |
| sentence-transformers | Dense vector encoder (all-MiniLM-L6-v2)   |
| faiss-cpu             | Vector similarity index (IndexFlatIP)     |
| google-generativeai   | Gemini LLM client for grounded generation |
| openai                | Alternative LLM client (GPT-4o-mini)      |
| numpy                 | Array handling for FAISS                  |

## 17. Conclusion

This report presented an end-to-end Retrieval-Augmented Generation system for research-paper question answering, built by Talla Tejaswini (231FA23034, 4th Year — Data Science). By combining page-aware PyMuPDF extraction, tunable sliding-window chunking, all-MiniLM-L6-v2 semantic embeddings, cosine-similarity search

over a FAISS IndexFlatIP index, and a strictly grounded Gemini/OpenAI generation step, the system turns a slow manual-reading task into an interactive question-answering experience — while keeping every answer traceable to an exact page in the source paper. The from-scratch, non-black-box implementation of every stage makes the design choices easy to explain, tune, and defend.

## 18. References

---

- Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 33.
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP.
- Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. IEEE Big Data.
- Google DeepMind. (2025). Gemini API Documentation.